

EXPERIMENT 6

Google Colab Pro Features — TPU/GPU Acceleration & Background Execution

Aim

To explore Google Colab's hardware acceleration (GPU/TPU), runtime management, and productivity features for training machine-learning models in the cloud.

Theory

Google Colaboratory (Colab) is a hosted Jupyter notebook environment that runs entirely in the browser and provides free access to GPUs and TPUs. Colab Pro / Pro+ offer faster GPUs (e.g. A100/V100), longer runtimes, more RAM and background execution so notebooks keep running after the browser tab is closed.

A GPU (Graphics Processing Unit) accelerates the massively parallel matrix operations at the heart of deep learning, while a TPU (Tensor Processing Unit) is Google's custom ASIC optimised specifically for tensor computation, offering higher throughput for large models. The runtime type is selected under Runtime → Change runtime type.

Colab mounts Google Drive for persistent storage (the VM's local disk is ephemeral). Files, datasets and model checkpoints should be saved to Drive because the runtime is reclaimed after idle timeouts.

Software / Tools Required

Google account, Google Colab (Pro recommended), a browser.

Procedure (Step-by-Step)

Step 1. Open colab.research.google.com and create a new notebook. Select GPU via Runtime → Change runtime type → Hardware accelerator → GPU.

```
# Verify the assigned GPU:  
!nvidia-smi
```

Step 2. Check that a deep-learning framework can see the GPU.

```
import torch  
print("CUDA available:", torch.cuda.is_available())  
print("Device:", torch.cuda.get_device_name(0))
```

Step 3. Benchmark a large matrix multiply on CPU vs GPU to see the speed-up.

```
import torch, time  
x = torch.randn(8000, 8000)  
t = time.time(); x @ x; print("CPU:", time.time()-t)  
xg = x.cuda()  
torch.cuda.synchronize(); t = time.time(); xg @ xg; torch.cuda.synchronize()  
print("GPU:", time.time()-t)
```

Step 4. Switch to TPU (Runtime → Change runtime type → TPU) and verify.

```
import os
```

```
| print("TPU address:", os.environ.get("COLAB_TPU_ADDR"))
```

Step 5. Mount Google Drive for persistent storage.

```
| from google.colab import drive  
| drive.mount("/content/drive")
```

Step 6. Save a trained model checkpoint to Drive so it survives runtime resets.

```
| import torch  
| os.makedirs("/content/drive/MyDrive/models", exist_ok=True)  
model_path = "/content/drive/MyDrive/models/demo.pt"  
  
| print("Checkpoint path:", model_path)
```

Note: add a test.txt file into drive in models folder

```
| !cat /content/drive/MyDrive/models/test.txt
```

Step 7. Monitor RAM and disk usage of the current runtime.

```
| !free -h  
| !df -h /content
```

Step 8. Upload a local file directly into the Colab session.

```
| from google.colab import files  
| uploaded = files.upload()
```

Step 9. Use shell magics to install extra packages inside the runtime.

```
| !pip install -q transformers accelerate
```

Step 10. Enable background execution (Colab Pro+) so a long training job continues after closing the tab, then reconnect later to view results.

```
| # Runtime -> Manage sessions shows active background runtimes
```

Expected Output

nvidia-smi lists the allocated GPU, torch reports CUDA is available, and the GPU matrix multiply is dramatically faster than the CPU one — confirming hardware acceleration is active. Drive mounts successfully for persistent storage.

Conclusion

Colab's GPU/TPU acceleration, Drive persistence and runtime-management features were explored, enabling cloud-based training of ML models without local hardware.

Viva Questions

1. What is the difference between a GPU and a TPU?
2. Why must datasets and checkpoints be saved to Google Drive in Colab?
3. What does 'torch.cuda.is_available()' check?
4. What advantages does Colab Pro/Pro+ add over free Colab?
5. What does the '!' prefix do in a Colab cell?